

PROPLEDGER ARCHITECTURE SERIES — VOLUME 03 OF 10

Advanced SQL & Analytics

CTEs, Window Functions, Complex Joins & Financial Reporting

DOMAIN LEVEL:

Enterprise Real Estate SaaS

Comparable System: Yardi Voyager / RealPage

FRAMEWORKS:

Spring Boot 3.3 | PG 16 | React 19

PAGE BUDGET:

10 Pages Complete Reference

1. DOCUMENT EXECUTIVE ABSTRACT & SCOPE

This volume deep-dives into the advanced PostgreSQL 16 SQL engineering powering PropLedger. It deconstructs production query designs for rent rolls, aged receivables, recursive asset rollups, and property P&L statements. Senior database engineering candidates will master Window Functions, Common Table Expressions, semi-joins, and execution optimization required to query millions of rows in single-digit milliseconds.

2. VOLUME SYLLABUS & CORE CAPABILITIES

Page	Chapter Title	Primary Technical Focus & Learning Outcome
02	Complex Multi-Table Joins	5-table joins across properties, units, leases, tenants, and ledger
03	Common Table Expressions (CTEs)	CTE pipeline design and explicit AS MATERIALIZED optimization
04	Correlated Subqueries & Semi-Joins	EXISTS vs IN execution plans and optimizer unnesting
05	Window Functions: Rent Roll	DENSE_RANK(), running cumulative sums, and partition variance
06	Occupancy Analytics	Calculating physical vs economic occupancy rates and loss-to-lease
07	Aging AR Delinquency Buckets	30/60/90+ day past due bucketing via FILTER (WHERE ...) clauses
08	Property P&L and NOI Statement	Synthesizing gross collected revenue against itemized OpEx categories
09	Recursive Hierarchy Rollups	WITH RECURSIVE rollups and JSONB state reconstruction queries
10	Top 15 SQL Interview Queries	High-yield real-world SQL problems asked at enterprise real estate tech

SENIOR STAFF ENGINEER REVIEWER GUIDANCE

This volume is engineered specifically for software engineering and database architecture interviews at Tier-1 real estate technology and enterprise SaaS corporations (Yardi, RealPage, AppFolio, CoStar). Every code sample reflects active, compilable implementations from the PropLedger production repository.

2. COMPLEX MULTI-TABLE JOINS IN PROPERTY OPERATIONS

5-TABLE OPERATIONAL JOINS & JOIN TYPE SELECTION

1. The Enterprise Rent Roll Query

In property management, a 'Rent Roll' is the definitive census report. It requires joining five relational boundaries: properties, buildings, units, leases, and primary tenants:

```
// CODE LISTING: SQL – MASTER OPERATIONAL RENT ROLL QUERY

-- 01_complex_joins.sql: Master Operational Rent Roll Census
SELECT
  p.property_code,
  p.name AS property_name,
  COALESCE(b.building_name, 'Main Structure') AS building,
  u.unit_number,
  u.unit_type,
  u.market_rent,
  u.status AS unit_status,
  l.lease_number,
  l.start_date,
  l.end_date,
  l.rent_amount AS contract_rent,
  t.first_name || ' ' || t.last_name AS primary_resident,
  t.email AS resident_email,
  COALESCE(SUM(i.balance_due), 0.00) AS total_past_due
FROM properties p
JOIN units u ON u.property_id = p.id
```

2. Join Semantics & Why LEFT JOIN is Critical Here

Notice that while 'JOIN units' is an INNER JOIN (every unit must have a property), the joins to 'leases', 'buildings', and 'tenants' must be ****LEFT JOINS****. An INNER JOIN would exclude all vacant apartments from the census, corrupting occupancy calculations!

3. COMMON TABLE EXPRESSIONS & MATERIALIZATION MECHANICS

POSTGRESQL 12+ CTE INLINING VS AS MATERIALIZED OPTIMIZATION

1. The Multi-Stage CTE Reporting Pipeline

Common Table Expressions (CTEs) break complex queries into readable, modular steps. In PropLedger's financial engine, we calculate tenant ledger standings through modular CTE pipelines:

```
// CODE LISTING: SQL - MODULAR COMMON TABLE EXPRESSION PIPELINE

-- 02_ctes_financial_reporting.sql: Multi-Stage Financial Delinquency CTE
WITH active_tenant_leases AS (
    SELECT l.id AS lease_id, l.unit_id, l.rent_amount, lt.tenant_id
    FROM leases l
    JOIN lease_tenants lt ON lt.lease_id = l.id AND lt.is_primary_tenant = TRUE
    WHERE l.status = 'ACTIVE'
),
tenant_balances AS (
    SELECT
        atl.tenant_id,
        atl.unit_id,
        SUM(i.total_amount) AS total_billed,
        SUM(i.balance_due) AS total_outstanding
    FROM active_tenant_leases atl
    JOIN invoices i ON i.lease_id = atl.lease_id
    GROUP BY atl.tenant_id, atl.unit_id
)
SELECT
```

2. 'AS MATERIALIZED' vs Inlined CTEs

In PostgreSQL 12+, the query planner inlines CTEs automatically. However, if a CTE is referenced multiple times, inlining can cause duplicate evaluations. Adding **AS MATERIALIZED** forces PostgreSQL to evaluate the CTE exactly once and cache the temporary working table in RAM.

4. CORRELATED SUBQUERIES VS SEMI-JOINS: PERFORMANCE

ANALYZING EXISTS VS IN VS LEFT JOIN ANTI-JOINS

1. The Vacant Units Problem: Three Query Approaches

Finding units that currently have zero active maintenance tickets or zero pending lease applications can be written in three distinct ways:

Approach	SQL Syntax	Planner Operator	Performance on Large Tables
NOT IN	WHERE id NOT IN (SELECT unit_id ...)	Seq Scan + Materialize	POOR (Fails on NULLs, O(N*M) worst case)
NOT EXISTS	WHERE NOT EXISTS (SELECT 1 ...)	Hash Anti-Join	EXCELLENT (Stops at first match, NULL ...)
LEFT JOIN + IS NULL	LEFT JOIN ... WHERE r.id IS NULL	Hash Anti-Join	EXCELLENT (Equivalent to NOT EXISTS)

2. Production Code: NOT EXISTS Anti-Join

```
// CODE LISTING: SQL – SEMI-JOIN ANTI-FILTER IMPLEMENTATION

-- 03_subqueries_correlated.sql: High-Performance Vacancy Query
SELECT
    u.id AS unit_id,
    u.unit_number,
    p.name AS property_name,
    u.market_rent
FROM units u
JOIN properties p ON p.id = u.property_id
WHERE u.status = 'AVAILABLE'
    AND NOT EXISTS (
        -- Semi-join check for any active lease covering today
        SELECT 1 FROM leases l
        WHERE l.unit_id = u.id
            AND l.status = 'ACTIVE'
            AND CURRENT_DATE BETWEEN l.start_date AND l.end_date
    )
    AND NOT EXISTS (
        -- Check for pending emergency work orders
```

INTERVIEW RULE OF THUMB

Never use 'NOT IN' with nullable foreign keys. In SQL 3-valued logic, if the subquery returns even a single NULL, 'NOT IN' evaluates to UNKNOWN, returning zero rows!

5. WINDOW FUNCTIONS: RENT ROLL & RUNNING TOTALS

OVER(), PARTITION BY, DENSE_RANK() & ANALYTIC AGGREGATIONS

1. Window Function Mechanics

Unlike 'GROUP BY' which collapses rows, SQL Window Functions compute aggregate values over a sliding partition while preserving the individual identity of every underlying row.

2. Production Code: Rent Roll Variance & Ranking

```
// CODE LISTING: SQL - ADVANCED WINDOW FUNCTIONS IMPLEMENTATION

-- 04_window_functions_rent_roll.sql: Rent Roll Analytics
SELECT
    p.name AS property_name,
    u.unit_number,
    u.bedrooms,
    l.rent_amount,
    -- 1. Average rent for this specific bedroom category across property
    ROUND(AVG(l.rent_amount) OVER (
        PARTITION BY p.id, u.bedrooms
    ), 2) AS avg_rent_for_beds,
    -- 2. Variance from bedroom average (Premium / Discount)
    ROUND(l.rent_amount - AVG(l.rent_amount) OVER (
        PARTITION BY p.id, u.bedrooms
    ), 2) AS variance_from_avg,
    -- 3. Dense Rank of rent within this building
    DENSE_RANK() OVER (
        PARTITION BY p.id ORDER BY l.rent_amount DESC
    ) AS rent_rank_in_property,
```

3. Key Function Explanations

- `AVG(...) OVER (PARTITION BY ...)`: Computes the sub-group benchmark without a separate self-join subquery.
- `DENSE_RANK()`: Ranks apartments by rent without skipping ranks in the event of identical rent amounts.
- `ROWS BETWEEN UNBOUNDED PRECEDING...`: Explicitly bounds the window frame for running accumulators.

6. OCCUPANCY RATE ANALYTICS & LOSS-TO-LEASE

PHYSICAL OCCUPANCY, ECONOMIC OCCUPANCY & FINANCIAL LEAKAGE

1. Physical vs Economic Occupancy

Institutional property investors track two distinct occupancy metrics:

- Physical Occupancy Rate: The percentage of physical units occupied by active leases: (Occupied Units / Total Units) * 100.
- Economic Occupancy Rate: The actual rent collected divided by the Gross Potential Rent (GPR) at full market rate.
- Loss-to-Lease: The financial gap between current market rents and contract rents signed in older leases.

2. Production Code: Occupancy Analytics

```
// CODE LISTING: SQL – COMPREHENSIVE OCCUPANCY METRICS QUERY
```

```
-- 05_occupancy_analytics.sql: Physical vs Economic Occupancy
```

```
SELECT
```

```
    p.id AS property_id,
```

```
    p.name AS property_name,
```

```
    COUNT(u.id) AS total_units,
```

```
    COUNT(CASE WHEN u.status = 'OCCUPIED' THEN 1 END) AS occupied_units,
```

```
    ROUND((COUNT(CASE WHEN u.status = 'OCCUPIED' THEN 1 END)::NUMERIC /  
            NULLIF(COUNT(u.id), 0)) * 100.0, 2) AS physical_occupancy_pct,
```

```
    -- Financial Potentials
```

```
    COALESCE(SUM(u.market_rent), 0.00) AS gross_potential_rent,
```

```
    COALESCE(SUM(l.rent_amount), 0.00) AS actual_contract_rent,
```

```
    -- Loss-to-Lease Calculation
```

```
    ROUND(COALESCE(SUM(u.market_rent), 0.00) - COALESCE(SUM(l.rent_amount), 0.00), 2) AS loss_to_lease,
```

```
    -- Economic Yield %
```

```
    ROUND((COALESCE(SUM(l.rent_amount), 0.00) /
```

```
            NULLIF(SUM(u.market_rent), 0)) * 100.0, 2) AS economic_occupancy_pct
```

```
FROM properties p
```

```
JOIN units u ON u.property_id = p.id
```

7. AGED RECEIVABLES (AGING AR) BUCKETING

CONDITIONAL AGGREGATIONS VIA FILTER (WHERE ...) CLAUSES

1. Accounts Receivable Aging Buckets

Delinquency analysis groups unpaid debt into standard aging buckets: Current, 1–30 Days, 31–60 Days, 61–90 Days, and 90+ Days Past Due. PropLedger implements this using PostgreSQL's modern 'FILTER (WHERE ...)' clause:

2. Production Code: Aging AR Query

```
// CODE LISTING: SQL – AGED ACCOUNTS RECEIVABLE IMPLEMENTATION

-- 06_aging_ar_analysis.sql: Aged Accounts Receivable Bucketing
SELECT
    p.name AS property_name,
    t.id AS tenant_id,
    t.first_name || ' ' || t.last_name AS resident_name,
    u.unit_number,
    SUM(i.balance_due) AS total_debt,
    -- Modern SQL: Filter Clauses
    COALESCE(SUM(i.balance_due) FILTER (
        WHERE i.due_date >= CURRENT_DATE
    ), 0.00) AS current_balance,
    COALESCE(SUM(i.balance_due) FILTER (
        WHERE CURRENT_DATE - i.due_date BETWEEN 1 AND 30
    ), 0.00) AS past_due_1_30,
    COALESCE(SUM(i.balance_due) FILTER (
        WHERE CURRENT_DATE - i.due_date BETWEEN 31 AND 60
    ), 0.00) AS past_due_31_60,
    COALESCE(SUM(i.balance_due) FILTER (
```

3. Performance Advantage of FILTER Over CASE WHEN

Standard 'SUM(CASE WHEN ... THEN balance_due ELSE 0 END)' must evaluate the CASE statement for every row. PostgreSQL's native 'FILTER (WHERE ...)' passes only matching tuples to the accumulator, executing significantly faster.

8. PROPERTY PROFIT & LOSS (P&L) AND NOI AGGREGATION

OPERATING EXPENSES, COLLECTED REVENUES & NET OPERATING INCOME

1. Net Operating Income (NOI) Financial Formula

NOI is the primary metric used by banks to value real estate. PropLedge synthesizes cash revenues against operating expenditures across all 13 accounting categories:

2. Production Code: Property P&L Statement

```
// CODE LISTING: SQL – PROPERTY PROFIT & LOSS STATEMENT
```

```
-- 07_property_pnl.sql: Comprehensive Property P&L
WITH collected_revenue AS (
    SELECT u.property_id, SUM(pa.allocated_amount) AS revenue
    FROM payment_allocations pa
    JOIN invoices i ON i.id = pa.invoice_id
    JOIN leases l ON l.id = i.lease_id
    JOIN units u ON u.id = l.unit_id
    WHERE pa.allocated_at >= '2026-01-01' AND pa.allocated_at < '2027-01-01'
    GROUP BY u.property_id
),
itemized_expenses AS (
    SELECT
        property_id,
        SUM(CASE WHEN category <> 'CAPITAL_IMPROVEMENT' THEN amount ELSE 0 END) AS total_opex,
        SUM(CASE WHEN category = 'CAPITAL_IMPROVEMENT' THEN amount ELSE 0 END) AS total_capex,
        SUM(CASE WHEN category = 'UTILITIES' THEN amount ELSE 0 END) AS opex_utilities,
        SUM(CASE WHEN category = 'REPAIRS' THEN amount ELSE 0 END) AS opex_repairs,
        SUM(CASE WHEN category = 'PROPERTY_TAX' THEN amount ELSE 0 END) AS opex_taxes
```

9. RECURSIVE HIERARCHIES & AUDIT RECONSTRUCTION

WITH RECURSIVE TREES & POINT-IN-TIME HISTORICAL DIFFS

1. Recursive Common Table Expressions (WITH RECURSIVE)

PropLedger supports nested asset hierarchies (Corporate Portfolio -> Regional Group -> Property -> Building -> Unit). Traversing this acyclic graph is executed via 'WITH RECURSIVE':

```
// CODE LISTING: SQL – RECURSIVE COMMON TABLE EXPRESSION

-- 09_recursive_hierarchies.sql: Recursive Asset Rollup
WITH RECURSIVE asset_tree AS (
  -- Anchor member: root properties
  SELECT id, name, NULL::BIGINT AS parent_id, 1 AS depth, name::TEXT AS path
  FROM properties WHERE is_active = TRUE
  UNION ALL
  -- Recursive member: buildings under properties
  SELECT b.id, b.building_name, b.property_id, at.depth + 1, at.path || ' -> ' || b.building_name
  FROM buildings b
  JOIN asset_tree at ON at.id = b.property_id
)
SELECT depth, path FROM asset_tree ORDER BY path;
```

2. Point-in-Time Audit State Reconstruction

Reconstructing the state of an invoice as of a specific second using JSONB deltas:

```
// CODE LISTING: SQL – JSONB AUDIT STATE RECONSTRUCTION

-- 10_audit_reconstruction.sql: Reconstruct Historical State
SELECT
  record_id, action, created_at AS transition_time,
  new_values->>'balance_due' AS balance_due_at_time,
  new_values->>'status' AS status_at_time,
  changed_by_user_id
FROM audit_logs
WHERE table_name = 'invoices' AND record_id = 104
  AND created_at <= '2026-06-01 12:00:00+00'
ORDER BY created_at DESC LIMIT 1;
```

10. TOP 15 SQL INTERVIEW QUERIES & SOLUTIONS

REAL-WORLD REAL ESTATE TECH INTERVIEW QUESTIONS

1. High-Yield Interview SQL Problems

Question 1: Find the 2nd highest contract rent in each property without using LIMIT/OFFSET.

Solution: Use DENSE_RANK() in a subquery: 'WITH ranked AS (SELECT rent_amount, property_id, DENSE_RANK() OVER (PARTITION BY property_id ORDER BY rent_amount DESC) as rnk FROM leases) SELECT * FROM ranked WHERE rnk = 2'. LIMIT/OFFSET cannot partition by property!

Question 2: How do you identify delinquent tenants whose debt has grown continuously for 3 consecutive months?

Solution: Use the LAG() window function: 'LAG(balance_due, 1) OVER (PARTITION BY tenant_id ORDER BY invoice_date)'. Compare 'balance_due > lag_1 AND lag_1 > lag_2' to detect strictly increasing arrears.

Question 3: Explain the difference between ROW_NUMBER(), RANK(), and DENSE_RANK().

Solution: Given tied rents (\$2000, \$2000, \$1800): ROW_NUMBER gives 1, 2, 3 (arbitrary tie-break). RANK gives 1, 1, 3 (skips rank 2). DENSE_RANK gives 1, 1, 2 (no gaps). For rent rolls, DENSE_RANK is preferred so tiers remain continuous.

SQL INTERVIEW PREPARATION TIP

Interviewers at enterprise software companies test your ability to avoid self-joins using Window Functions and your understanding of NULL handling in NOT IN vs NOT EXISTS subqueries.